Luke Weinbach (solo)

luke.weinbach@yale,edu

A YOLOv8 Model for Litter Detection

1. Introduction

On November 15 of this year, after over two years and more than $24 million spent, the city of Columbia, South Carolina (my hometown) will host the official reopening ceremony of Finlay Park in the downtown area. The park's reopening represents a great step forward for the promotion of quality green spaces in my home city. I also believe that the opening of a big city park project in 2025 brings with it a great opportunity to put Columbia, SC at the forefront of experimentation with the future of AI-enhanced Smart Park Management systems. To that end, I would like to potentially contribute one small piece of that, starting with a litter detection program that uses computer vision to identify trash around the park.

One can imagine a system in the not-so-distant future where park security cameras do a sweep before the park opens for the day, marks areas where litter is detected, and flags those areas to the maintenance crew to make park cleanup much faster and easier. You could also imagine a vision system inside a trash-collecting robot that roams the park collecting trash throughout the day. I hope to train a model that might be used to help identify trash in either or both of these systems.

I approach this problem by implementing an open source YOLOv8 bounding box object detection model, training on primarily open-source data supplemented lightly with some specially curated data from on the premises at two City of Columbia parks—Finlay Park and Riverwalk. In general, the biggest issue with this project and many like it is data quality. Using an open source dataset with over 3,000 images means there are possibilities for incorrectly-labeled data to affect the performance of the model.

2. Related Work

This project is implemented under the YOLO (You Only Look Once) architecture framework as originally described in [1]. The YOLO family of models is particularly well-suited for a task like litter detection due to its single-stage detection architecture. Unlike two-stage

detectors (such as Faster R-CNN) that first generate region proposals and then classify them, YOLO reframes object detection as a single-pass regression problem, mapping image pixels directly to bounding box coordinates and class probabilities.

I implement a version using the open source eighth version, YOLOv8 [2]. YOLOv8 introduces several architectural and training refinements over predecessors like YOLOv5 and YOLOv7, significantly enhancing both detection accuracy and convergence speed. One such advancement is that YOLOv8 employs a task-aligned assigner to ensure that high confidence scores are strictly correlated with accurate bounding box coordinates. This reduces the prevalence of poorly localized targets, which is huge for this particular project. If the outlined goal of this project is to be able to notify staff of a park about where trash is, then we can see that this model is not very helpful if it can see trash *anywhere* within the security camera's FOV but not tell the staff where exactly to go to collect it. The localization process is additionally improved by employing a decoupled head architecture (i.e., processing classification and regression independently). Furthermore, the YOLOv8 training process utilizes a composite loss strategy involving Distribution Focal Loss (DFL) and Complete Intersection over Union (CIoU) loss. DFL improves performance on crushed or occluded litter by robustly handling ambiguous object boundaries, while CIoU minimizes center-point distance and aspect ratio errors, ensuring the vision system provides reliable spatial.

Adjustments from the standard-install YOLOv8 configuration include a switch to the AdamW optimizer [3], which I implement based on prior work showing that Adam-based optimizers increase accuracy and reduce the time to convergence when applied to YOLOv8 object detection use cases [4]. I also decided to add a partial backbone freeze to the network. "Freezing" refers to the process of locking specific layers of a neural network so their weights are not updated during training. By preventing backpropagation through these layers, the model retains the robust, low-level feature extractors (such as edge and texture detectors) learned from a massive pre-training dataset like COCO. [5] demonstrates that this not only reduces computational cost and memory usage but prevents the destruction of the fundamental visual filters learned in pretraining (in other words, prevents overfitting to this single application).

3. Methodology

I adapt a pre-trained general object detector to the specific, often ambiguous domain of trash identification through transfer learning and rigorous data simplification. I selected the YOLOv8 (You Only Look Once, version 8) architecture developed by Ultralytics for its state-of-the-art balance between inference speed and detection accuracy, which is critical for the proposed application of real-time processing on security camera feeds or mobile robots. While I initially prototyped with the smaller YOLOv8n (Nano) model, I ultimately deployed the YOLOv8m (Medium) variant. The Medium model offers higher feature capacity, which proved necessary for distinguishing small, irregular trash objects from the various park backgrounds and object scales where the Nano model struggled to generalize. This switch ultimately led to small but significant accuracy gains.

The model architecture utilizes the standard YOLOv8 compound loss function, comprising Complete IoU (CIoU) and Distribution Focal Loss (DFL) for bounding box regression, alongside Binary Cross-Entropy (BCE) for classification. While the default YOLOv8 configuration typically employs Stochastic Gradient Descent (SGD) with linear decay, I opted to switch the optimizer to AdamW [3] and enabled Cosine Annealing for the learning rate schedule. I selected AdamW to leverage its decoupled weight decay, which allows for faster convergence on sparse features (typical of small objects) while effectively mitigating overfitting compared to standard SGD. This optimization was paired with a Cosine Annealing learning rate scheduler to ensure smooth convergence into a robust local minimum.

The model is coded and run within a Google Colab notebook, using the Ultralytics module to download YOLOv8 and Roboflow the API to download data. Training is executed in a runtime using L4 GPU hardware.

4. Data

I take the lion's share of the data (3,501 images) from an open source dataset called *trash detect* uploaded by user Dune to the computer vision platform Roboflow Universe [6]. This data comes prelabeled with bounding boxes and is labeled in six categories, corresponding to different types of litter. Note that the type of litter is not relevant in this project, since my only

goal is to map instances of any trash and notify park staff. Therefore, I have preprocessed this data to unify all categories into a single category—'trash'—effectively simplifying the objective function from a multi-class classification problem to a binary object detection problem (Trash vs. Background), which significantly improved model convergence.

Additionally, to see if my model would perform well in its intended industry solution environment, I collected a very modest-sized amount of data on-site in city parks:

- I went to Finlay Park myself, bringing a few commonly littered objects (soda cans, styrofoam, plastic bottles, and paper products [magazine/cardboard containers, etc.]). I placed these objects one at a time in different places around the park and snapped 4-8 photos of each one, taking about one step back in the direction of the nearest park security camera after each photo in order to capture the object from various distances (trying to induce scale-invariance from the data side).[1] I collected 60 total images using this method.

Example 1 – image taken from nearby an object

Example 2 – Image of the same object pictured in Example 1 from a farther distance

- I also contacted my City Recreation Commission for help. They were able to provide me with five images of large quantities of litter collected in a single area taken by park rangers along the Saluda Riverwalk. These photos each contained upwards of 40-60 littered objects in frame. This data will present an interesting test of this model on its capability to label many littered items in the same shot.

[1] Yes, I picked up every piece of trash after photographing it!

Example 1 – Large collection of littered trash

Example 2 – Large collection of littered trash

The default configuration of the dataset autogenerates at most three versions of each image in the set with various filters and/or transformations applied. These include:

- Reflection across the y axis
- 90º Rotations (clockwise, counterclockwise, & upside down)
- Cropping (0-20% zoom)
- Rotation (±45º)
- Shear (±12º vertical, ±12º horizontal)
- Noise (added noise to up to 1.5% of pixels)

Note: I would later on in this project come to find out that the annotation quality of this dataset was extremely poor, which contributed to the relative failure in the final results. In fact, cleaning the data turns out to be the most important kind of "pseudo-hyperparameter tuning" and *certainly* the most bang-for-your-buck use of tuning time. See Sections 6 and 7 for more on this issue.

5. Implementation Details

- *Data Preprocessing.* This data runs through the standard YOLOv8 data preprocessing pipeline, which automatically standardizes input data before training. The pipeline first applies auto-orientation to correct image rotation based on EXIF metadata. Subsequently, a letterbox resizing technique is utilized to scale the longest dimension

of each image to the target resolution. This method preserves the original aspect ratio by padding the remaining areas with gray pixels to generate a square, thereby preventing distortion of the objects. Finally, pixel intensity values are normalized from their native 0–255 integer range to a 0.0–1.0 floating-point scale to ensure numerical stability and facilitate efficient convergence.

- *Training Setup.* Training requires us to download the model framework from the Ultralytics python package and the dataset from the Roboflow API. Note that downloading the dataset requires a Roboflow API key. Instructions on how to acquire one for free can be found in the Colab notebook.
- *Base Model.* The default parameter setup for this project YOLOv8 is defined as the following. A training run with these parameters is run exclusively on the open source dataset (without my own data included) in order to set a performance baseline:
    - Epochs: 50
    - Image Size: 640x640
    - Batch Size: 16
    - Optimizer: Stochastic Gradient Descent
    - Learning Rate: 0.01
    - Learning Rate Scheduler: Linear Decay
- *Improved (Tuned) Model.* I tested trial runs with many different hyperparameter configurations and architectural details backed by research. Ultimately, I determined that the best possible configuration was:
    - Epochs: 20
        - AdamW is a much more aggressive optimizer. In my tests, the best model waits would usually be achieved somewhere between epochs 9-15. 20 epochs is an appropriate number that ensures every training run gets to its optimal weights while not spending too much time training in circles without improving.
    - Image Size: 1280x1280
        - Many of the images from the dataset contain instances of very small litter objects. 640x640 images make these objects far too small for a model to learn them in a truly effective manner.

- Batch Size: 8
    - Because I increased image size twofold, each image now contains 4x as many pixels as before. I decreased the batch size to help optimize and maintain memory performance.
- Optimizer: AdamW
    - I switched to an AdamW optimizer based on the conclusions of [4] that demonstrate the optimizer's performance advantages over SGD for object detection tasks.
- Learning Rate: 0.001
    - This is also a downstream effect based the shift to AdamW. To combat the aggressive nature of AdamW's changes, I reduce the starting learning rate by a factor of 10 as a counterbalance.
- Learning Rate Scheduler: Cosine Annealing
    - As opposed to linear decay which adjusts the learning rate abruptly, cosine annealing lowers the learning rate in a smooth curve. With a relatively small (by vision standards) dataset that included many small, semi unique objects, smoothing out the decay of the learning rate prevents overfitting to specific examples.
- Freeze: 10 layers
    - [5] finds 10 to be the optimal value for freeze in fine-grained detection tasks. This isolates layers 0-9 of the network and prevents them from being altered by training. These early layers detect basic lines, edges, and simple textures, and since "edges" look the same in a park as they do in the COCO dataset, there is no need to retrain them. Freezing them saves memory and prevents the model from "forgetting" these basics.
- Close Mosaic: final 10 epochs
    - YOLO automatically stitches images together during training to make "mosaic images" that are great for training a model to recognize objects at different scales, as frequently mosaics stitch images with small object together with those containing larger objects. However,

this process also introduces artificial borders and unnatural contexts. In order to get the scale advantages of mosaic, I allow it for the first 10 epochs. Then I set 'close_mosaic' to 10 so that the training protocol avoids mosaic stitching for the last 10, allowing us to recover from any effects of the unnatural borders and contexts induced by the mosaics.

6. Results

The base model run of training (using exclusively open source data for training, evaluation, and testing) with minimal hyperparameter tuning was able to achieve a mAP50 score of **0.511**.[2] I also check the performance of this run of training on a separate test set that includes instances of my own collected data, where an mAP50 of **0.42** is attained. This gives us a decent heuristic for measuring how well the open source data matches the data collected in my exact intended use case. The match is far from exact—simply adding my own data in with the test batch reduces accuracy by 32%. From this, it is clear that for the most optimal implementation of this system at scale, as much of the data as possible should come from on-site at the location of implementation. We can also get some insight into why this is happening through the other performance metrics—namely, recall. Recall essentially measures how well the model does at detecting multiple instances of litter in the same image. The recall for the base model performance on the dataset altered by adding my own data came in at an exceptionally low **0.304**. This might indicate that the Riverwalk images may be causing a lot of the accuracy issues in the test data. Even though they only represent a few images, they represent a ton of class labels, as the images average 45 labeled instances of trash per example. The recall metric shows that the model is vastly underperforming on examples with multiple objects present in a single image, which naturally leads me to believe that the examples with a ton of labeled objects may generally be the culprit for tanking the performance on a set with this added data.

[2] mAP50 (mean Average Precision at 0.50 IoU [Intersection over Union]) is a standard metric for evaluating object detection performance. It calculates the average precision across all classes, considering a prediction "correct" only if the predicted bounding box overlaps with the true box by at least 50%.

I ease the problem of dataset cross-compatibility by including some of the Finlay and Riverwalk data in the training and evaluation sets. Ultimately I decide to split the groups as follows:

Location	# in Training Set	# in Evaluation Set	# in Test Set	Total
Finlay Park	40	10	10	60
Riverwalk	2	1	2	5

This is the dataset I run for the rest of the training experimentation.

I then move on to tuning the hyperparameters, the final versions of which can be viewed in Section 5. Training with these improved all performance metrics, though only slightly:

Precision	Recall	mAP50	mAP50-95
0.698	0.534	0.552	0.384

The hyperparameter tuning improved performance marginally, but ultimately, model performance stalled out. I narrowed the final issue down to data quality. In light of this, I took a small subset of 200 images from a relatively even split across training, validation, and test set and corrected their labeling. Rerunning the fine-tuned model on this slightly improved data returned *massive* performance dividends:

Precision	Recall	mAP50	mAP50-95
0.729	0.622	0.667	0.468

To get a summary understanding of how much this improved performance relative to the finetuning along and how much overall performance improved over the whole process, I have pooled all the results into the table on the following page as well as some example outputs.

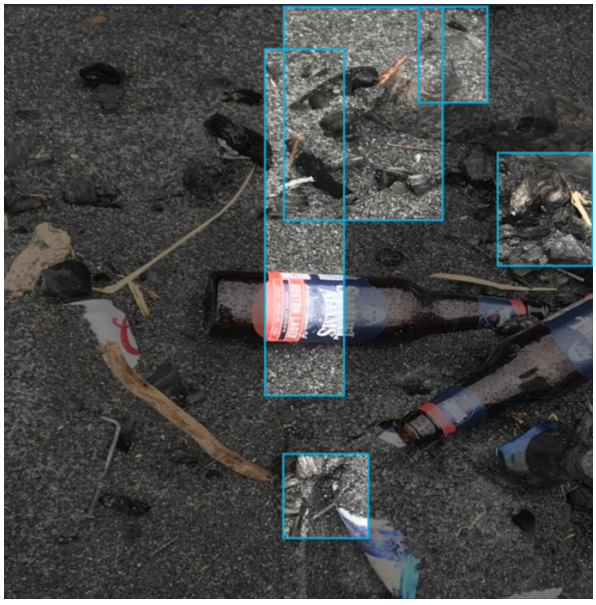
Model	Precision	Recall	mAP50	mAP50-95
Original	0.655	0.500	0.511	0.352
Fine-Tuned	0.698	0.534	0.552	0.384

Fine-Tuned (w/ improved data)	0.729	0.622	0.667	0.468

Example outputs from the validation set of our final model, trained with optimal hyperparameters, data from all sources, and 200 corrected examples from the original dataset

7. Discussion

The golden lesson to be taken from this project is an understanding of the paramount importance of good data for high-performance industrial vision applications. An intense suite of research- and test-backed hyperparameter tuning only improved accuracy by a couple percentage points, while correcting the labeling on a small portion of the examples yielded huge performance gains. The single most prominent challenge during the implementation of this project and the ultimate single limitation of this study both boil down wholly to the data. Many examples from the original open source dataset were either partially or totally incorrect in labelling. For example:



Example image taken from the dataset. The bounding boxes are completely dissociated from the objects they are intended to bound—instead, they mostly bound dirt.

Example image taken from the dataset, the bounding box covers up nearly an entire quarter of the screen—and yet that quarter contains no trash at all.

After examining a sufficient chunk of the data, I roughly estimate that anywhere between 15-40% of this set is of similarly poor annotation quality. It was infeasible to parse the entire dataset, identify all the errors, and correct them in the time frame of this project. It is also not tenable to train a high accuracy model on scarce data when a non-insignificant portion of it looks like this.

However, there are still many wins I take from the completion of this project. First, I was able to iteratively and experimentally narrow down the exact roadblock that was keeping performance low (data). This allowed me to make great strides near the end of the project toward the more accurate model I had originally conceived. Additionally, the goal of hyperparameter

fine tuning is to eke out the last few percentage points of accuracy after the basic training regimen is set up. I was successful in this as well, since I managed to bring mAP50 up from 0.511 to 0.552 through fine tuning alone. If applied to a model with completely fixed data, I'm confident these hyperparameters would still be near-optimal.

In summary, while I was not quite successfully able to train a model that accomplished my original 70% goal for precision, I am extremely confident that I correctly identified the major roadblock that prevented my model from reaching that goal and that I will be able to further develop this model to a standard that suffices and even surpasses my original plans for it. Clearly then, given more time the obvious action to be taken for improved performance would be to go through and confirm or correct the bounding box drawings of every single example from the original dataset.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *arXiv*, Jun. 2015, arxiv.org/abs/1506.02640.

[2] G. Jocher, J. Qiu, & A. Chaurasia, "Ultralytics YOLO (Version 8.0.0)" [Computer software], 2023, github.com/ultralytics/ultralytics.

[3] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *arXiv*, Nov. 2017, arxiv.org/abs/1711.05101.

[4] M.M. Bhosale and Y.S. Angal, "Performance analysis of momentum of Adam optimizer on YOLO-V8 using traffic object dataset," *Traitment du Signal,* vol. 42, no. 1, pp. 505-518, Feb 2025, doi: 10.18280/ts.420143.

[5] V. Gandhi and S. Gandhi, "Fine-Tuning Without Forgetting: Adaptation of YOLOv8 Preserves COCO Performance," *arXiv*, May 2025, arxiv.org/abs/2505.01016.

[6] Dune, 'trash detect Dataset'. Roboflow Universe, Mar. 2025.